ENCM 515 – Lab 1

Group #	Part A Group 23	Date: Feb 5th
Student name(s)	Lee Evans	
	Sudarshan Naicker	
Instructor/TA		

To submit:

- **Add your lab sheet (as PDF if possible) and your main.c to a zip archive and upload to D2L.**

PART ONE

LEDs

LD3 – colour: ORANGE – I/O Port: PD13

LD4 – colour: GREEN – I/O Port: PD12

LD5 – colour: RED – I/O Port: PD14

LD6 – colour: BLUE – I/O Port: PD15

Push Button

B1 – I/O Port: PA0

B2 – RESET, I/O Port: NRST

Q1: What is the base address of the peripheral space?

```
#define PERIPH_BASE           0x40000000UL /*!< Peripheral base address in the alias region
```

From stm32f411xe.h

Q2: What are the functions for each of the buttons A – H?

A- Terminate and Relaunch

B- Resume

C- Suspend

D-Terminate

E- Disconnect

F- Step Into

G- Step Over

H- Step Return

Q3: How much time is elapsed between each WRITE? Is this expected?

Aside from the first write which happens once the system builds and runs, we get roughly 1 second. The fact that it isnt precise on 1 second makes sense, assuming the timer that comes configured for us is EXACTLY accurate on 1000ms, we are also processing lines with printf() and if() which take time from the microcontroller in addition to our 1s delay.

Console | Problems | Executables | Debugger Console | Memory | SWV ITM Data Console | SWV Data Trace

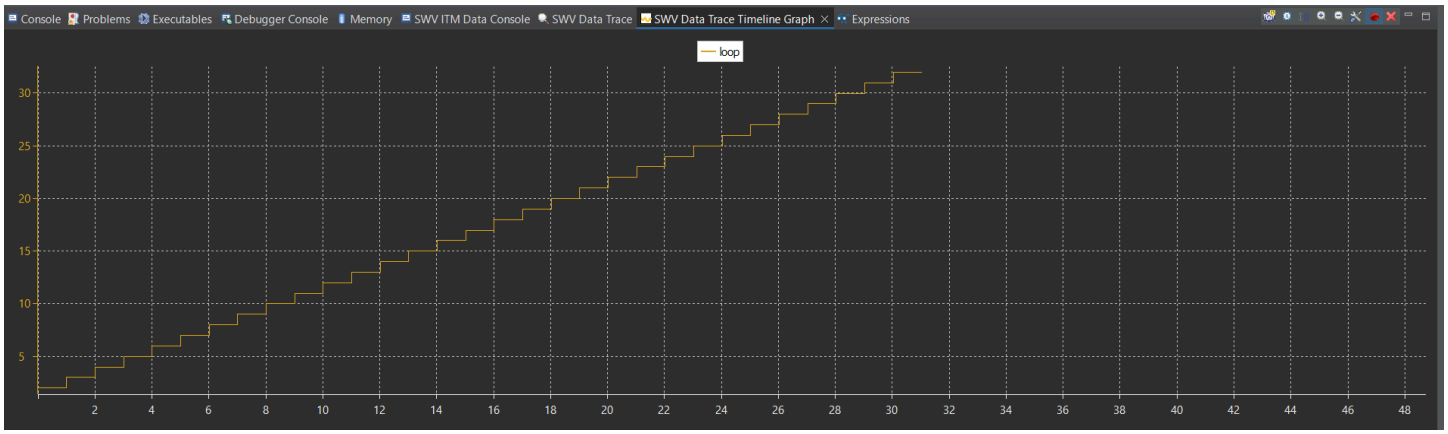
Watch

Comp	Name	Value
0	loop	15

History (loop)

Access	Value	PC	Cycles	Time
WRITE	1		57886	602.979167 µs
WRITE	2		96153848	1.001603 s
WRITE	3		192249785	2.002602 s
WRITE	4		288345739	3.003601 s
WRITE	5		384441676	4.004601 s
WRITE	6		480537633	5.005600 s
WRITE	7		576633568	6.006600 s
WRITE	8		672729527	7.007599 s
WRITE	9		768825469	8.008599 s
WRITE	10		864921406	9.009598 s
WRITE	11		961017360	10.010598 s
WRITE	12		1057113304	11.011597 s
WRITE	13		1153209250	12.012596 s
WRITE	14		1249305194	13.013596 s
WRITE	15		1345401148	14.014595 s

Q4: Take a screenshot of the SWV Data Trace Timeline Graph. You might need to wait some time to get an interesting shape to appear. Alternatively, adjust your delay so that the Loop value is incremented more often.



Q5: Copy the listing for the `test_assembly` function and annotate it. Mark which lines of assembly correspond to what you've written and what corresponds to the compiler-inserted code for the Calling Convention. Explain what's going on (e.g., here we can see the registers that will be overwritten are being saved on the stack…. Now we see them restored…)

We used ChatGPT for deciphering the assembly code line by line and to better understand exactly what was going on.

58 loop = test_assembly(10, 15); // here is the line we call the function

0800059e: movs r1, #15 //The two `movs` lines below appear to be storing our a and b input values from the function call to registers r0 and r1 respectively.

080005a0: movs r0, #10

080005a2: bl 0x800054e <test_assembly> // bl is jumping to the line in memory where our function is stored

test_assembly: // we then jump to the memory space where our test assembly function is defined

0800054e: push {r7} // register value in r7 gets stored in memory

08000550: sub sp, #20 // This next line is allocating 20 bytes for the following operations

08000552: add r7, sp, #0 //store the value of sp in register r7

08000554: str r0, [r7, #4] // Storing r1 and r2 on the stack. this is are our values of a nad b.

08000556: str r1, [r7, #0]

30 int res = 0; // assign a new variable to a value of 0

08000558: movs r3, #0 // sets r3 to 0

0800055a: str r3, [r7, #12] //

31 __asm volatile ("ADD %[result], %[input_i], %[input_b]"

0800055c: ldr r3, [r7, #4] //load a from r7+4 and store in r3

0800055e: ldr r2, [r7, #0] //load b from r7+0 and store in r2

08000560: add r3, r2 // r3 = r3+ r2

08000562: str r3, [r7, #12] // store r3 in r7+12 (after adding the a and b)

35 return res;

08000564: ldr r3, [r7, #12]

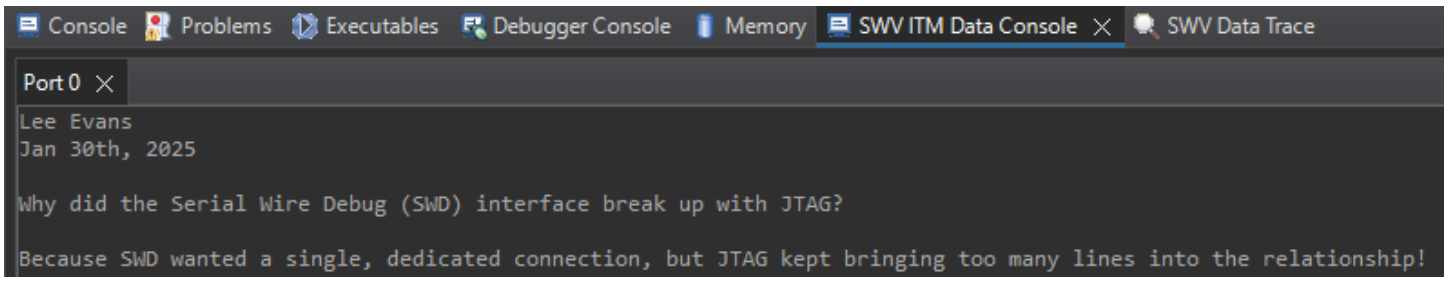
36 }

080005a6: mov r3, r0 // moves the return values in r0 to r3

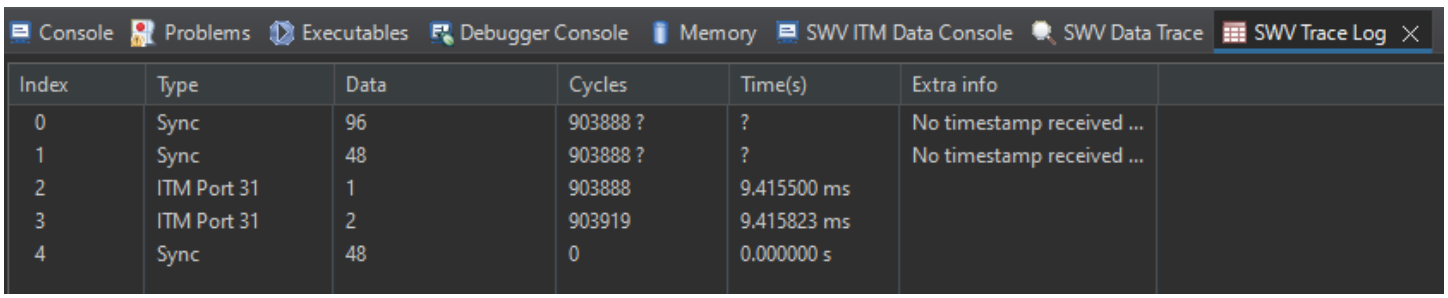
080005a8: strb r3, [r7, #7]

PART TWO

Q1> Screenshot of the console output (name, date, joke)



Q2> Time stamps and cycles for the two ITM Port 31 entries (screenshot should suffice).



Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	96	903888 ?	?	No timestamp received ...
1	Sync	48	903888 ?	?	No timestamp received ...
2	ITM Port 31	1	903888	9.415500 ms	
3	ITM Port 31	2	903919	9.415823 ms	
4	Sync	48	0	0.000000 s	

How many clock cycles are there between the two writes to the ITM Port 31?

clock cycles = 903919 - 903888 = 31

How much time has elapsed?

dt = 9.415823 - 9.415500 = 0.000323

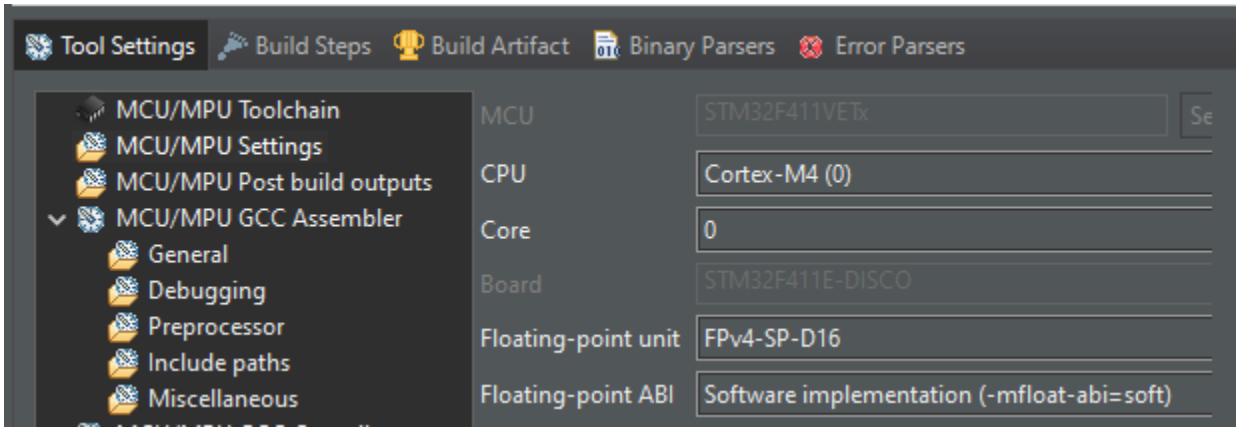
What is the average amount of time taken for each loop iteration?

sec per loop = 0.000323/2000000 = 0.0000000001615

Q3 > > How many clock cycles are taken to perform the floating point function?

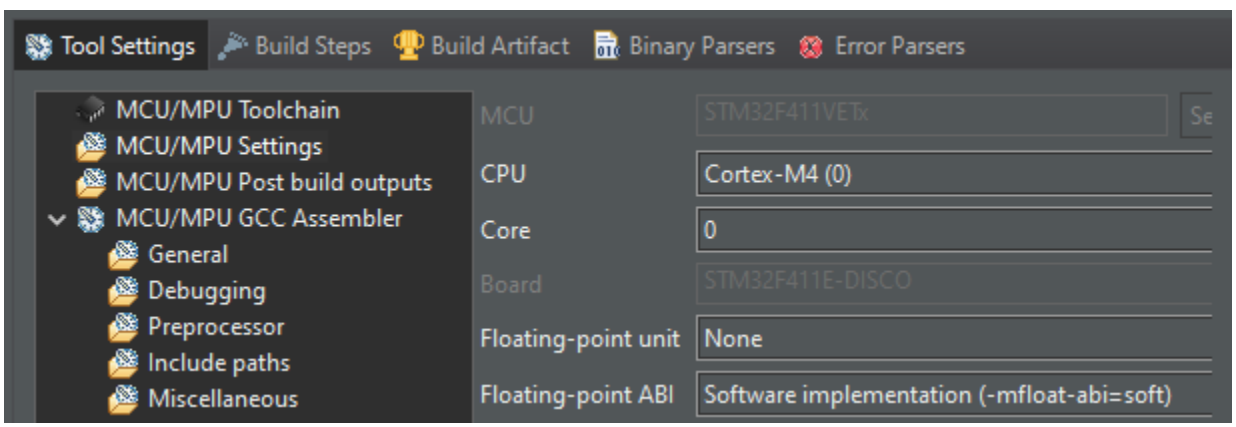
Test	Value of A	Value of B	Clock Cycles needed
1	0.5	0.125	31
2	5.4	0.125	35
3	10.8	0.2	38
4	0.2	0.0125	38
5	100.8	0.4	38

80 cycles with FPU disabled and multiplying 100.8 x 0.4.



Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	96	302672 ?	?	No timestamp received ...
1	Sync	48	302672 ?	?	No timestamp received ...
2	ITM Port 31	1	302672	3.152833 ms	
3	ITM Port 31	2	302752	3.153667 ms	
4	Sync	48	0	0.000000 s	

Q4 > How many clock cycles are taken to perform the floating point function now? Include a screenshot of the SWV Trace Log.



Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	96	1836052 ?	?	No timestamp received ...
1	Sync	48	1836052 ?	?	No timestamp received ...
2	ITM Port 31	1	1836052	19.125542 ms	
3	ITM Port 31	2	1836132	19.126375 ms	
4	Sync	48	0	0.000000 s	

80 cycles after disabling the FPU and multiplying 100.8 x 0.4

Q5 > Fix the code snippet so that the correct values are loaded in a and b. Paste your completed function here.

Below shows the q1.15 representation of o.5 and 0.125 loaded into a and b variables.

```
void FixedExperiment(void) {
      int16_t a = 16384;   // should be 0.5; + 0.100000000000000 = 16384 in base 10
      int16_t b = 4096; // should be 0.125; 0.001000000000000 = 4096 in base 10
      int16_t c = a * b; // leave this as is initially
}
```

Name	Type	Value
a	int16_t	16384
b	int16_t	4096
c	int16_t	0

After leaving the multiplication operation as is, variable C gets the value 0 as seen in the image above from the variables window. This makes sense since our C variable is only 16 bits. Our value is getting truncated and we are losing the MSBs.

a	int16_t	16384
b	int16_t	4096
c	int32_t	67108864

After adjusting variable C to hold 32 bit values, we see the correct value of 67108864. We can now shift the result 15 bits so C can store the proper value as an int16_t

```
void FixedExperiment(void) {
    int16_t a = 16384;   // should be 0.5; + 0.100000000000000 = 16384 in base 10
    int16_t b = 4096; // should be 0.125; 0.001000000000000 = 4096 in base 10
    int32_t temp = ((int32_t)a * (int32_t)b);
    int16_t c = temp>>15; // leave this as is initially
}
```

Name	Type	Value
a	int16_t	16384
b	int16_t	4096
temp	int32_t	67108864
c	int16_t	2048

As you can see above, the int value of 2048 is correct. When converted back to a decimal value from q1.15, the result is 0.0625.

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	104	1985280 ?	?	No timestamp rece...
1	ITM Port 31	1	1985280	20.680000 ...	
2	Sync	48	1985327 ?	?	No timestamp rece...
3	Sync	48	1985327 ?	?	No timestamp rece...
4	Sync	48	1985327 ?	?	No timestamp rece...
5	ITM Port 31	2	1985327	20.680490 ...	
6	Sync	48	0	0.000000 s	

From the image above we can see that it took 47 cycles to compute the result.

Below is the final function for fixed point multiplication.

```
void FixedExperiment(void) {
	int16_t a = 16384;   // should be 0.5; + 0.100000000000000 = 16384 in base 10
	int16_t b = 4096; // should be 0.125; 0.001000000000000 = 4096 in base 10
	int32_t temp = ((int32_t)a * (int32_t)b);
	int16_t c = temp>>15; // leave this as is initially
}
```

Q6> Take a screenshot of the Variables window after calling these functions. What do you observe?

Name	Type	Value
floatArray	float32_t [10]	0x2001ffc0
floatArray[0]	float32_t	0.5
floatArray[1]	float32_t	-0.300000012
floatArray[2]	float32_t	1.20000005
floatArray[3]	float32_t	-1.10000002
floatArray[4]	float32_t	0.99000001
floatArray[5]	float32_t	-0.99000001
floatArray[6]	float32_t	0.75
floatArray[7]	float32_t	-0.449999988
floatArray[8]	float32_t	1.5
floatArray[9]	float32_t	-1.5

Name	Type	Value
q15Array	q15_t [10]	0x2001ffac
q15Array[0]	q15_t	16384
q15Array[1]	q15_t	-9830
q15Array[2]	q15_t	32767
q15Array[3]	q15_t	-32768
q15Array[4]	q15_t	32440
q15Array[5]	q15_t	-32440
q15Array[6]	q15_t	24576
q15Array[7]	q15_t	-14745
q15Array[8]	q15_t	32767
q15Array[9]	q15_t	-32768

convertedFloatArray	float32_t [10]	0x2001ff84
convertedFloatArray[0]	float32_t	0.5
convertedFloatArray[1]	float32_t	-0.299987793
convertedFloatArray[2]	float32_t	0.999969482
convertedFloatArray[3]	float32_t	-1
convertedFloatArray[4]	float32_t	0.989990234
convertedFloatArray[5]	float32_t	-0.989990234
convertedFloatArray[6]	float32_t	0.75
convertedFloatArray[7]	float32_t	-0.449981689
convertedFloatArray[8]	float32_t	0.999969482
convertedFloatArray[9]	float32_t	-1

What we notice from switching float to q1.15 format and back to float, is that information is lost. This makes sense because fixed point format loses information during rounding, while floating point is more accurate. Additionally, we see that any values we converted outside the range of [1,-1], have been lost. since q1.15 format can only do values between [1,-1], our values that were outside this range now are the limits. We would need to use another fix point format to properly convert these values.

Q7> Write a reflection of what you have observed and learned in this lab [approx.. 200 words max.]. Which lecture concepts did you encounter here?

1. First and foremost, we learned how to use STMCube and some of its niche features for debugging and monitoring whats happening under the hood. It was interesting to see how easily one can view the generated assembly code.
2. Paying attention to the operations you do is important since each line of code requires a certain amount of cycles. Depending on the application this could be crucial to track!
3. Floating point units are very beneficial! Being able to avoid converting to fixed point format using built in hardware saves on design complexity.
4. We learned how to convert floats to q15 format, which was also in the lecture 2025_02_04_A Gentle Intro to DSP IV_A, and were also able to see first hand how data can be lost through fixed point math operations. Overall this lab heavily leveraged the lecture content on fixed and floating point operations.